

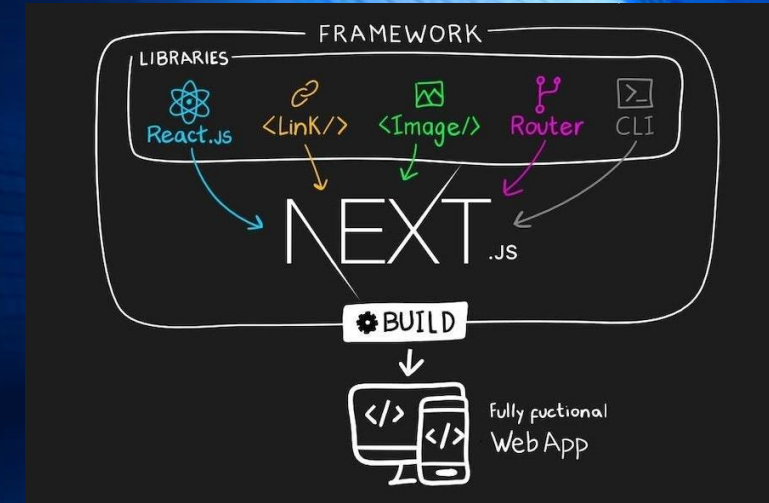
CLASS ASSIGNMENT 4

Name: Zunaira Hussain

Day: Thursday

Slot: 9-12

Teachers: Sir Hamza Alvi and
Sir Fahad Khan

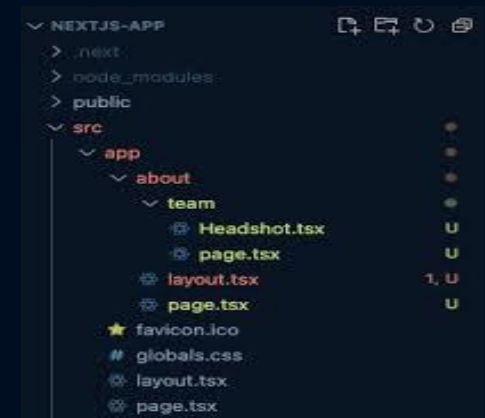


WHAT IS NEXT.JS ?

- Next.js is a powerful framework for building React applications that enhances performance, scalability, and developer experience. Here's a detailed and beautiful exploration of its three key points:
- **1. Server-Side Rendering (SSR) and Static Site Generation (SSG)** Next.js excels in providing both server-side rendering (SSR) and static site generation (SSG), allowing developers to choose the best rendering strategy for their needs.
- **2. File-Based Routing** Next.js simplifies routing with a file-based system, which enhances organization and developer productivity.
- **3. API Routes** Next.js includes a built-in API routes feature that allows developers to create backend functionality within their applications.



PAGE.TSX



Page.tsx defines individual pages and their unique content. Each file within the pages directory corresponds to a route in your application.

Structure: The Page.tsx files focus on the unique content and layout of individual pages, such as text, images, and other elements relevant to that specific route.

/pages

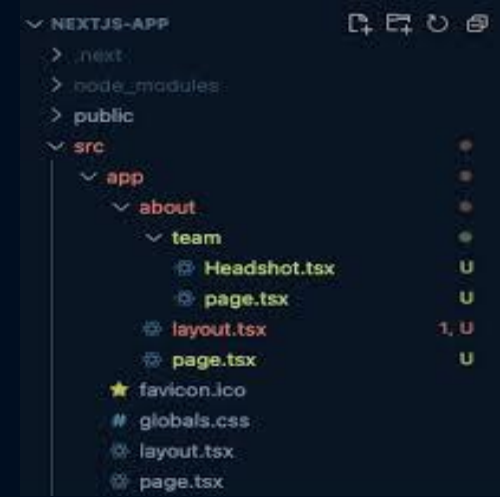
_app.tsx

index.tsx

/app

layout.tsx

page.tsx



LAYOUT.TSX

`layout.tsx` provides a common structure and UI elements shared across those pages.

The `layout.tsx` file is used to define a common layout structure that can be shared across multiple pages.

`layout.tsx` component renders once for each page, wrapping the content of `page.tsx`.

WHAT IS THE LINK TAG??

- The `<Link>` component is part of the Next.js framework and is designed to enhance the user experience by enabling client-side transitions between routes. Unlike traditional anchor (`<a>`) tags, which cause a full page reload, the `<Link>` component allows for faster navigation by leveraging the power of client-side routing.

- **PURPOSE:**

- The `<Link>` tag in Next.js enables seamless client-side navigation between pages, enhancing user experience by allowing instant transitions without full page reloads.
- It preloads linked pages for faster access and automatically handles accessibility concerns.
- This makes it essential for creating responsive and dynamic web applications.



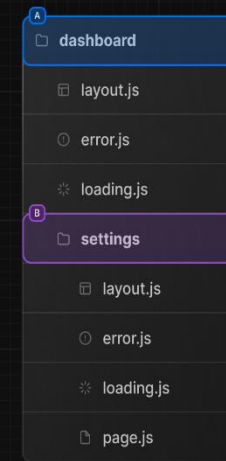
`<Link />`



```
1 // index.js
2 import Link from 'next/link';
3
4 export default () => (
5   <div>
6     Hello World.{ ' }
7     <Link href='/about'>
8       <a>About</a>
9     </Link>
10   </div>
11 );
12
```

HOW WE CREATE NESTED PAGES??IN NEXT JS

- Creating nested pages in Next.js is simple! Here are the steps:
- Create a Directory: Inside the pages folder, create a new folder for your nested .
- Add a File: Inside the folder, create a file for the main page .
- Create Nested Pages: Add additional files for nested pages within the folder .
- Define the Component: In each file, define a React component to render the content for that page.
- Use <Link> for Navigation: Use the <Link> component to link to your nested pages from other parts of your app.
- **Access Nested Pages:**
 - Navigate to the nested pages using their respective URLs for the main page
 - Test in Browser: Run your Next.js app and ensure all nested routes function as expected!



COMPONENTS AND ITS USAGE

- Components are reusable building blocks that help create the user interface of your application.
- Components are self-contained pieces of UI that can manage their own state and logic.
- They can be simple (like buttons or input fields) or complex (like entire sections of a page).
- Next.js, built on React, encourages the use of components to promote modularity and reusability.



CSS IN NEXT JS

- **Here are five ways to apply CSS in Next.js**
- **:Global CSS:** Create a CSS file (e.g., styles/globals.css) and import it in pages/_app.tsx to apply styles globally.
- **CSS Modules:** Create a CSS file with the .module.css extension (e.g., styles/Home.module.css) and import it in your component.
- **Tailwind CSS:** Install Tailwind CSS and configure it, then use utility classes directly in your TSX.
- **Styled Components:** Install styled-components library, then create styled components in your React files.



TAILWIND CSS VS STANDARD CSS

STYLES:

- Tailwind CSS: Utilizes a utility-first approach, where predefined classes are applied directly in the HTML to style elements. This promotes rapid development and consistency by reusing utility classes.
- Standard CSS: Involves writing custom styles in separate CSS files, where styles are typically defined in selectors and applied to HTML elements using class or ID attributes.

Customization and Flexibility

Tailwind CSS: Offers extensive customization through a configuration file (`tailwind.config.js`), allowing developers to define their design system, including colors, spacing, and typography, while maintaining a consistent look across the application.

- Standard CSS: Provides full control over styles but can lead to inconsistencies if not managed carefully, as styles may become scattered across different files or components.

THANK YOU SO MUCH
I HOPE YOU LIKE MY PRESENTATION